

Aether — Intelligent Workload Scheduling

Right Workload, Right Runtime, Right Time

Four scheduling strategies, resource-aware placement, constraint-based filtering, and optimization suggestions — across Podman, Kubernetes, KubeVirt, and Metal3.

Balanced • Cost • Performance • Bin-Packing

Scheduling Strategies

Aether's scheduler evaluates every runtime against your workload requirements and selects the optimal placement based on the active strategy.



Balanced (Default)

Weights both cost and performance equally. Allocates 30% scoring weight to cost efficiency and 30% to resource headroom, producing well-rounded placements that avoid extremes.



Cost Optimized

Minimizes estimated daily cost by favoring runtimes with the lowest cost-per-CPU and cost-per-GB rates. Podman typically wins for small workloads at \$0.50/CPU/day vs Metal3 at \$3.00/CPU/day.



Performance Optimized

Maximizes resource headroom by favoring runtimes with the lowest utilization after placement. Spreads workloads across runtimes to prevent contention and noisy-neighbor effects.



Bin Packing

Packs workloads tightly onto the fewest runtimes possible. Maximizes utilization on active runtimes, allowing unused runtimes to be powered down for infrastructure savings.

```
# Schedule with a specific strategy
$ aether schedule my-app --strategy cost
$ aether schedule my-app --strategy performance
$ aether schedule my-app --strategy bin-packing
$ aether schedule my-app --strategy balanced
```



The scheduler scores each candidate runtime from 0.0 to 1.0, combining strategy weight (60%), preferred runtime bonus (20%), affinity score (15%), and capacity headroom (10%).

Resource-Aware Placement

Every scheduling decision considers CPU, memory, storage, GPU requirements, and runtime-specific constraints before selecting a target.

Runtime Capacity Model

Runtime	CPU	Memory	GPU	Cost/CPU/Day	Max Workloads
Podman	8 cores	16 GB	0	\$0.50	20
Kubernetes	64 cores	128 GB	2	\$1.00	200
KubeVirt	32 cores	64 GB	4	\$2.00	50
Metal3	128 cores	512 GB	8	\$3.00	20

Constraint-Based Filtering



Runtime Constraints

`RequireRuntime` and `ExcludeRuntime` constraints hard-filter the candidate list before scoring begins. Co-locate and anti-affinity constraints link placements between workloads.



Hardware Constraints

`RequireGpu` filters to runtimes with available GPUs. `RequireBareMetal` restricts to Metal3 only. `MaxCostPerDay` caps the estimated daily cost per placement.

```
# Schedule with constraints
$ aether schedule gpu-trainer \
  --cpu 8 --memory 32768 --gpu 2 \
  --constraint require:gpu \
  --constraint zone:us-east-1a \
  --preferred-runtime kubevirt
```

Utilization Monitoring

Real-time visibility into resource utilization across all four runtimes. Identify overloaded runtimes, underused capacity, and cost-saving opportunities.

```
$ aether schedule utilization
```

Runtime Utilization:

Runtime	CPU	Memory	Workloads	Cost/Day
Podman	25%	25%	2/20	\$ 1.20
Kubernetes	25%	25%	10/200	\$ 12.80
KubeVirt	25%	25%	3/50	\$ 6.40
Metal3	25%	25%	1/20	\$ 9.60

Utilization Metrics



CPU Utilization

Calculated as $(\text{total_cpu} - \text{available_cpu}) / \text{total_cpu}$.
Tracked per-runtime and used by the Performance strategy to spread workloads and by the Bin Packing strategy to consolidate them.



Memory Utilization

Calculated as $(\text{total_memory} - \text{available_memory}) / \text{total_memory}$.
Combined with CPU utilization to produce a composite utilization score that drives placement decisions.



Cost Tracking

Each runtime tracks cost-per-CPU-day and cost-per-GB-day rates. The scheduler estimates daily cost



Capacity Warnings

When a placement pushes utilization above 85%, the scheduler emits a warning.

for each placement candidate and surfaces total runtime cost in the utilization summary.

Approaching the max workload limit also triggers a warning to prevent capacity exhaustion.

Optimization Suggestions

The scheduler continuously analyzes placement patterns and generates actionable recommendations to reduce cost, improve balance, and prevent capacity issues.

```
$ aether schedule optimize
```

```
[CAPACITY] Kubernetes CPU utilization at 92% - consider scaling up  
or migrating workloads
```

```
[COST] Metal3 is underutilized (5% CPU) with 1 workload -  
consider consolidation  
Potential saving: $4.80/day
```

```
[COST] 'data-loader' uses only 2 CPUs on Metal3 - consider  
moving to Kubernetes or Podman  
Potential saving: $4.00/day
```

```
[BALANCE] KubeVirt utilization (75%) deviates significantly from  
average (40%)
```

Optimization Categories

Category	Trigger	Action
CAPACITY	CPU utilization > 90%	Scale up runtime or migrate workloads out
COST	CPU utilization < 10% with active workloads	Consolidate to cheaper runtime
COST	Small workload on expensive runtime	Move workloads using <4 CPUs off Metal3
BALANCE	Utilization deviates >30% from average	Redistribute workloads across runtimes

Placement History

Every scheduling decision is recorded as a **Placement** record with workload name, selected runtime, reserved CPU, reserved memory, and RFC 3339 timestamp. When a workload is deleted, `release()` frees the reserved capacity back to the runtime. This history enables capacity planning, chargeback reporting, and audit trail compliance.

```
$ aether schedule history
```

Workload	Runtime	CPU	Memory	Placed At
web-frontend	Podman	2.0	2048 MB	2026-04-10T08:00:00Z
api-service	Kubernetes	4.0	8192 MB	2026-04-10T09:15:00Z
gpu-trainer	KubeVirt	8.0	32 GB	2026-04-11T14:30:00Z
data-pipeline	Metal3	16.0	64 GB	2026-04-12T06:00:00Z

Schedule Smarter

Let Aether's scheduler place workloads where they belong — automatically, efficiently, and with full transparency.

4

Scheduling Strategies

8

Constraint Types

4

Optimization Categories

Intelligent Placement for Every Workload

From cost-optimized dev workloads to performance-critical production services, Aether schedules across Podman, Kubernetes, KubeVirt, and Metal3 with a single command.

```
# End-to-end scheduling workflow
$ aether schedule my-app --cpu 4 --memory 8192 --strategy balanced
$ aether schedule utilization
$ aether schedule optimize
$ aether schedule history
```